

時刻変化するワークロード に対するスキーマ最適化

11/7 B4 増田莞爾

背景

- 従来のスキーマ設計は、ワークロードが静的であると仮定することが多い[1][2]
 - スキーマとは、データベース内のテーブルや実体化ビューの集合
- 多くのアプリケーションでは、周期的・進化的など、動的で予測可能なワークロードパターンが見られる
これは、IoTや科学的分析において、クエリの順序が事前に分かっている場合によく見られる [3]

背景

予測可能なワークロードの例：天文データ解析の例

日中

利用者：日本の研究チーム

研究テーマ：天の川銀河内の「恒星」

実行されるタスク：特定の恒星の明るさ
変化等

恒星データに特化した検索



夜間

利用者：ヨーロッパの研究チーム

研究テーマ：遠方の「銀河」

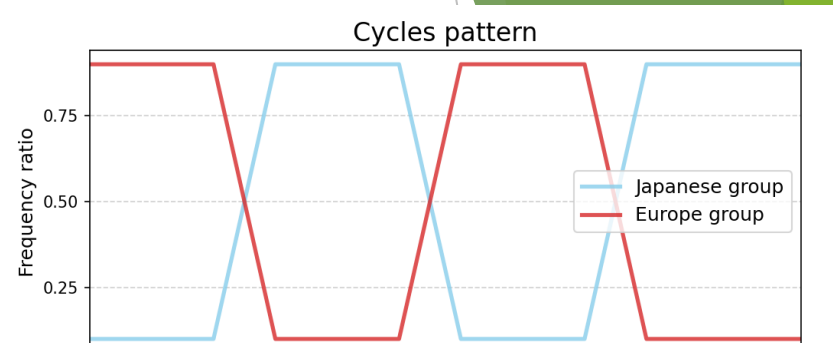
実行されるタスク：遠方銀河の形状分
析等

銀河データに特化した検索

同じデータベースでも時間帯によって検索対象の天体や条件が変わり
実行されるクエリの種類が予測可能に変化する

研究目的：時刻変化するワークロードに対して、時系列スキーマを最適化すること

3



事前知識

- ▶ PostgreSQLにおける**実体化ビュー(Materialized View)**
 - クエリの結果をデータとして物理的に保存する特殊なテーブル
 - 複雑な集計やテーブル結合など、時間のかかるクエリを事前に実行しておける
 - 参照時は計算済みの結果を返すだけなので高速

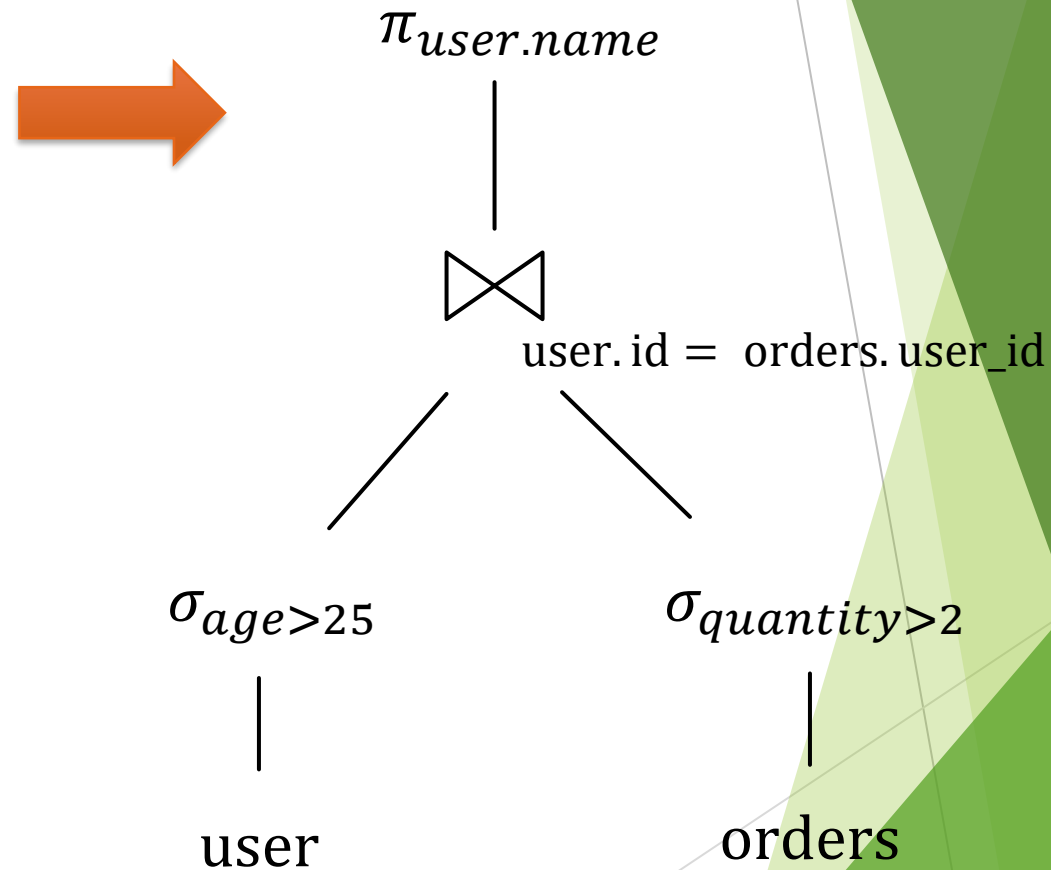
以下のクエリについての実行プランを考える

```
SELECT user.name  
FROM user  
JOIN orders ON user.id = orders.user_id  
WHERE user.age > 25 AND orders.quantity > 2
```

事前知識

```
SELECT user.name  
FROM user  
JOIN orders ON user.id = orders.user_id  
WHERE user.age > 25 AND orders.quantity > 2
```

実体化ビューなしだとJOINを
含め全ての操作を毎回実行する必要あり
→ クエリの実行コストは高く**低速**



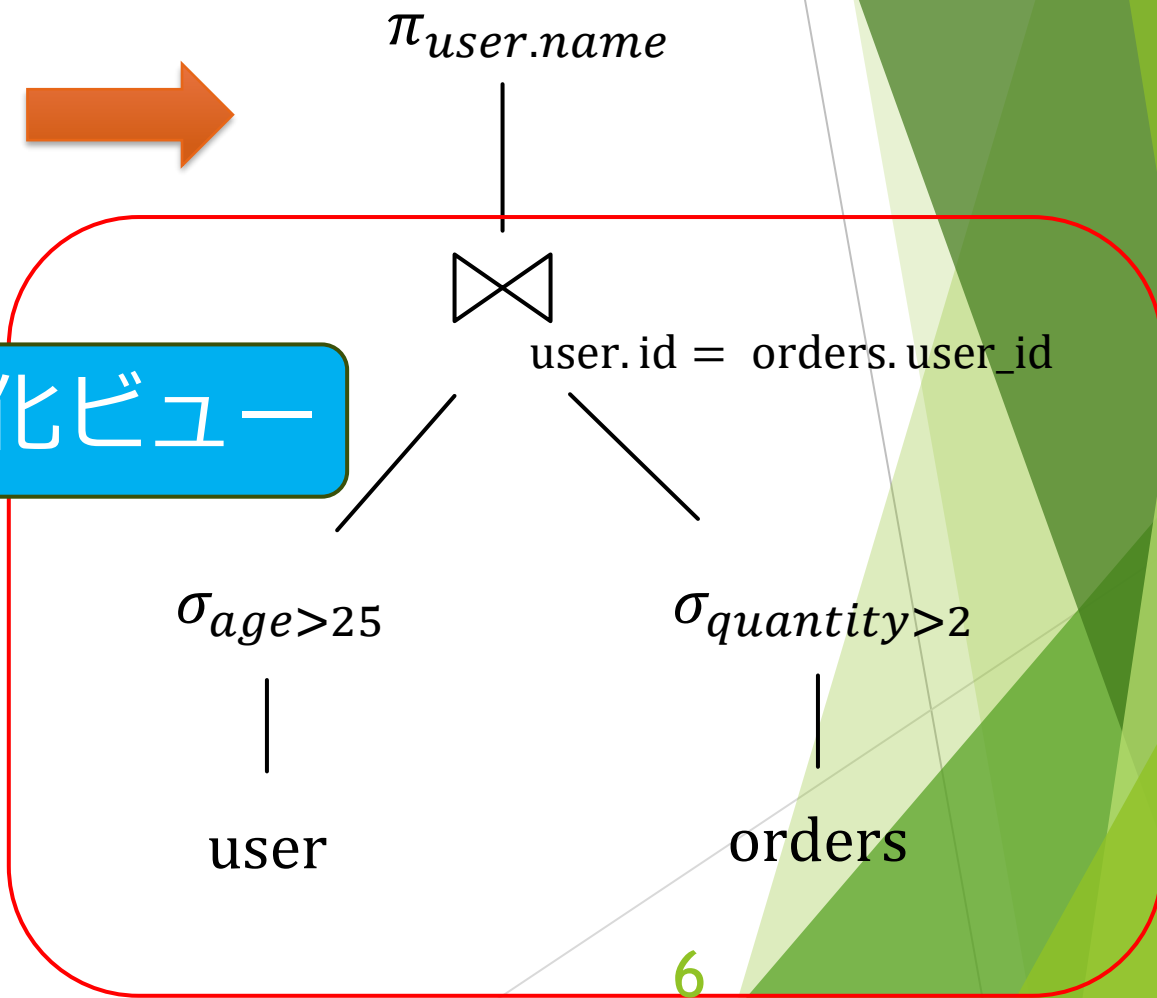
事前知識

```
SELECT user.name  
FROM user  
JOIN orders ON user.id = orders.user_id  
WHERE user.age > 25 AND orders.quantity > 2
```

実体化ビューありだとJOINまでの操作
は行わなくて良い

→ クエリの実行コストが下がり**高速**

実体化ビュー



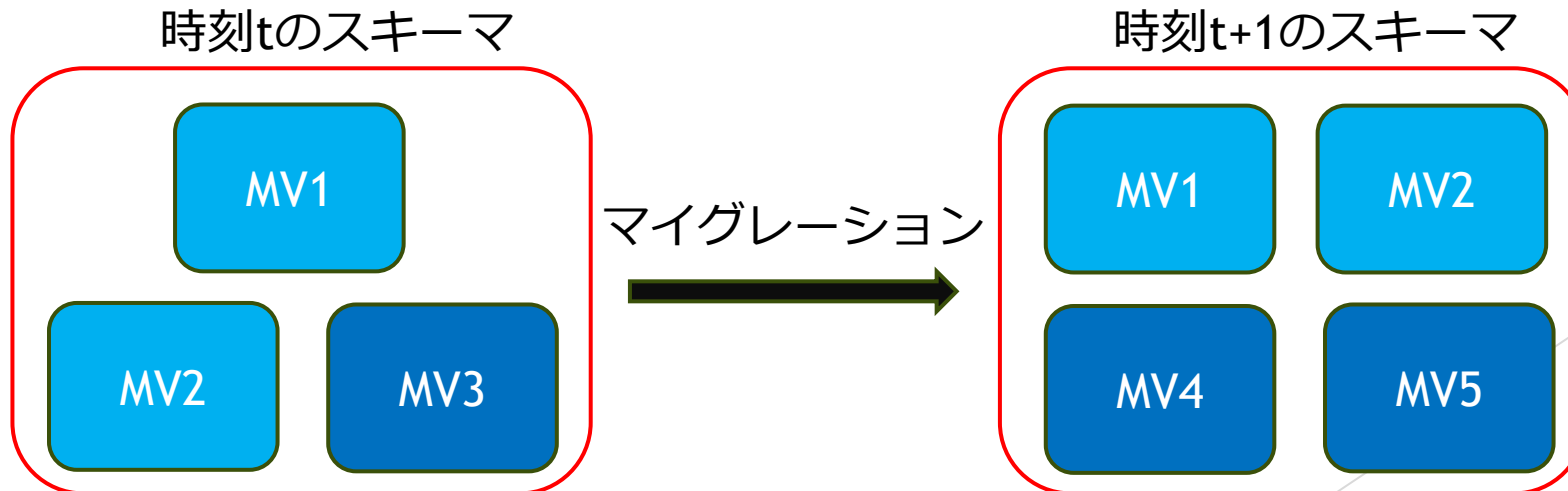
事前知識

▶ ワークロード実行コスト

- ▶ ワークロードに含まれるクエリを実行するコスト
- ▶ 適切な実体化ビューを利用することで小さくすることが出来る

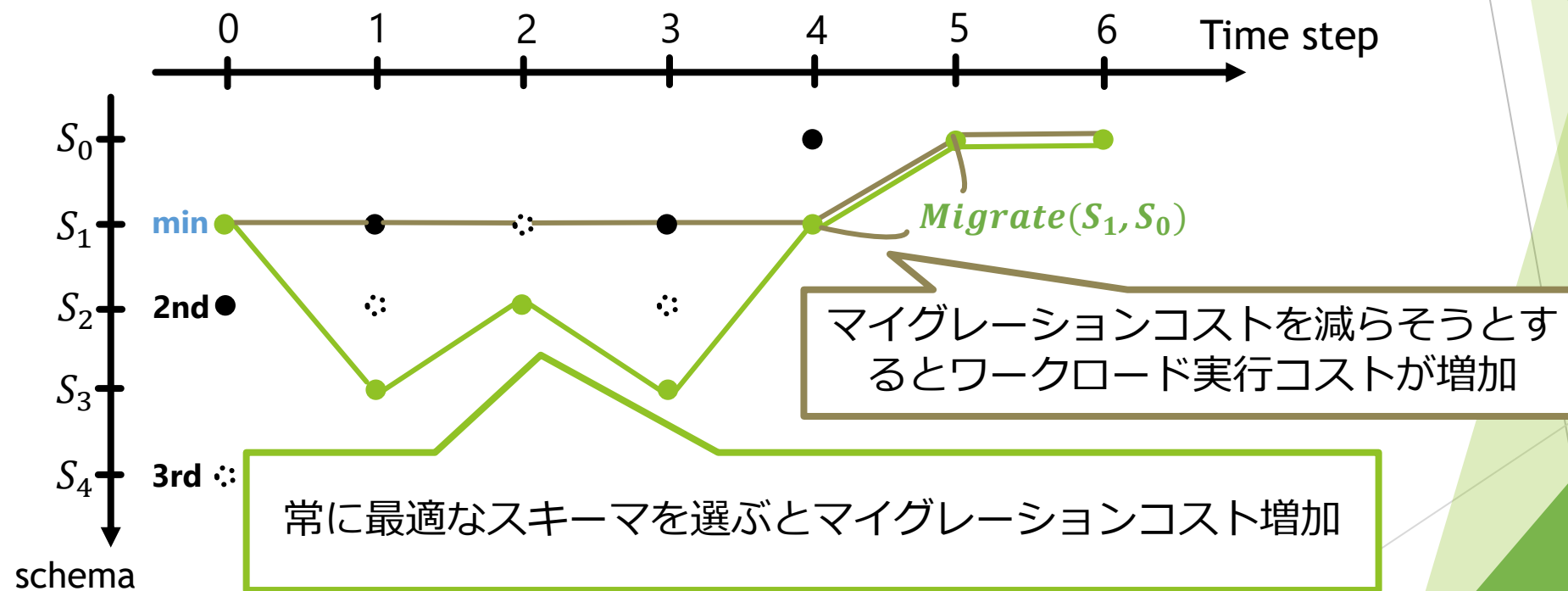
▶ マイグレーションコスト

- ▶ 各時刻間で選ばれるスキーマが変わるとき、次の時刻での実体化ビューを作成する必要がある
- ▶ このときにかかるコストをマイグレーションコストと呼ぶ



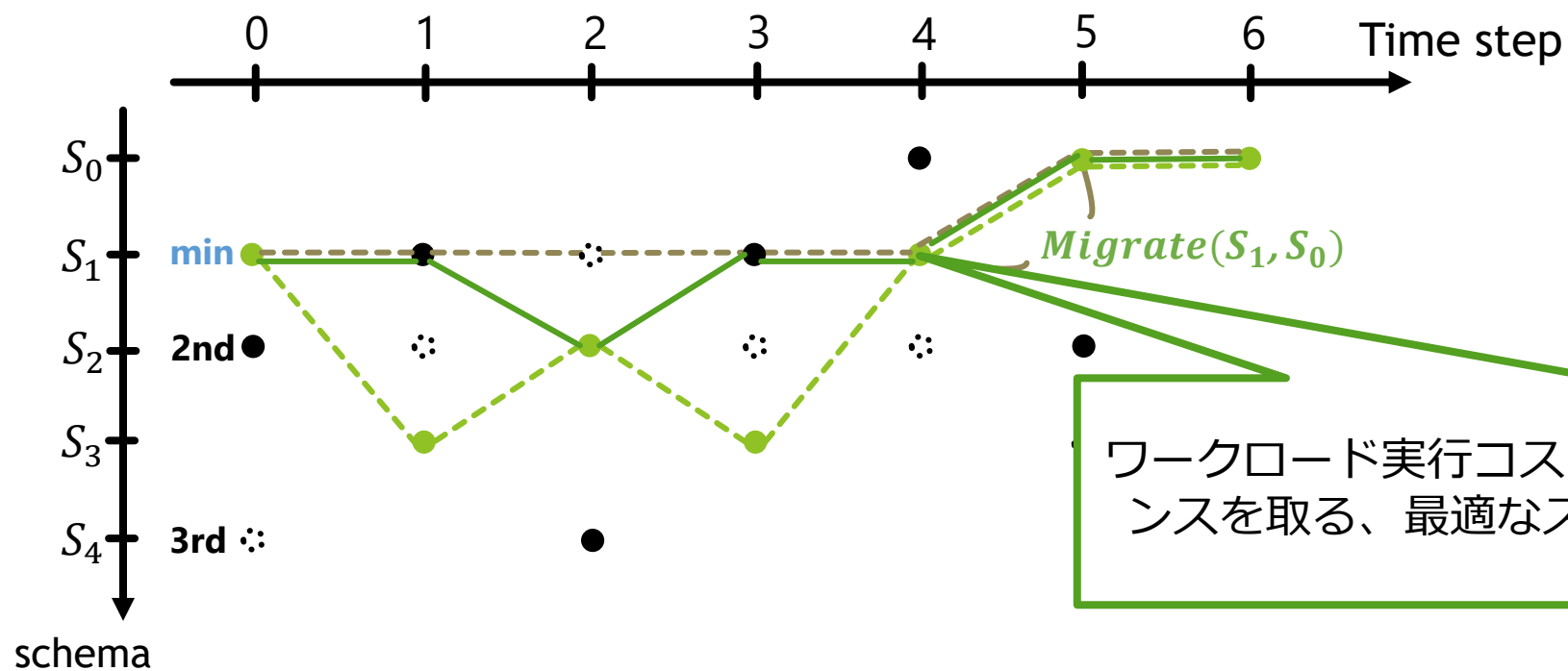
課題

- 時刻変化するワークロードでは、最適なスキーマは時刻に応じて変化する



課題

- ▶ スキーマ選択の際は、ワークロード実行コストとマイグレーションコストのトレードオフを考える必要がある



ワークロード実行コストとマイグレーションコストのバランスを取る、最適なスキーマシーケンスが存在するはず

課題まとめ

▶ 目的

- ▶ 時刻変化するワークロードに対して、実行コストとマイグレーションコストの総和を最小化するスキーマを提案する

▶ 解決しなければならない課題

- ▶ ワークロード実行コストとマイグレーションコストのトレードオフ
 - ▶ 整数線形計画問題(ILP)として定式化
- ▶ 膨大なクエリ数とタイムステップが、最適化モデルの変数の規模を増大させる
 - ▶ ワークロードを再帰的に抽象化してスキーマ候補を枝刈りし、最適化を高速化

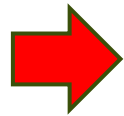
提案手法

入力

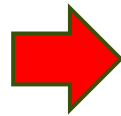
時刻変化する
ワークロード

スキーマ

容量制約



提案手法



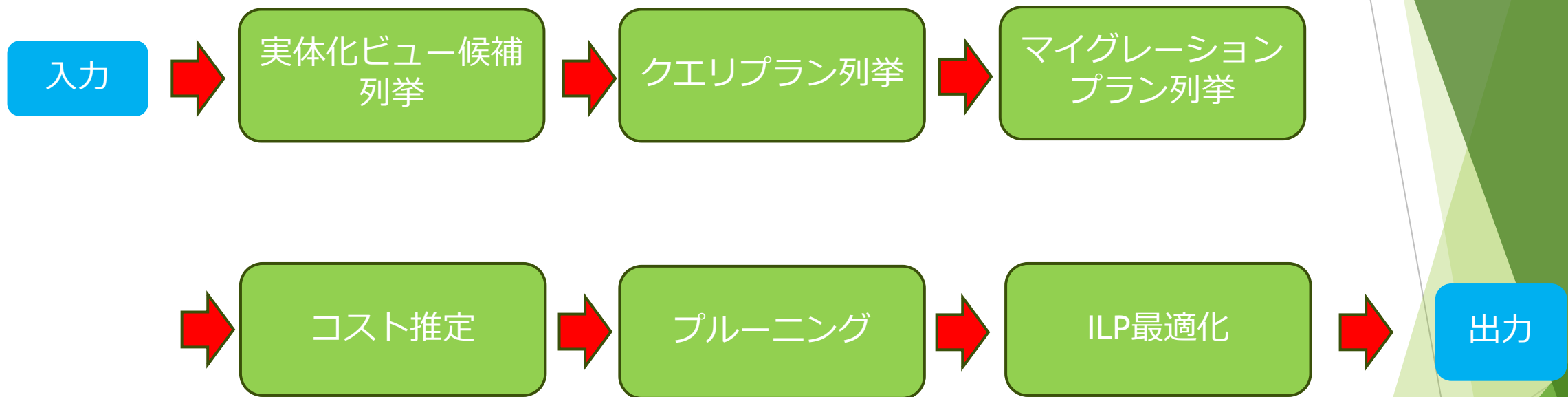
出力

各時刻のスキーマ

各時刻のクエリプラン

各時刻間のマイグレーションプラン

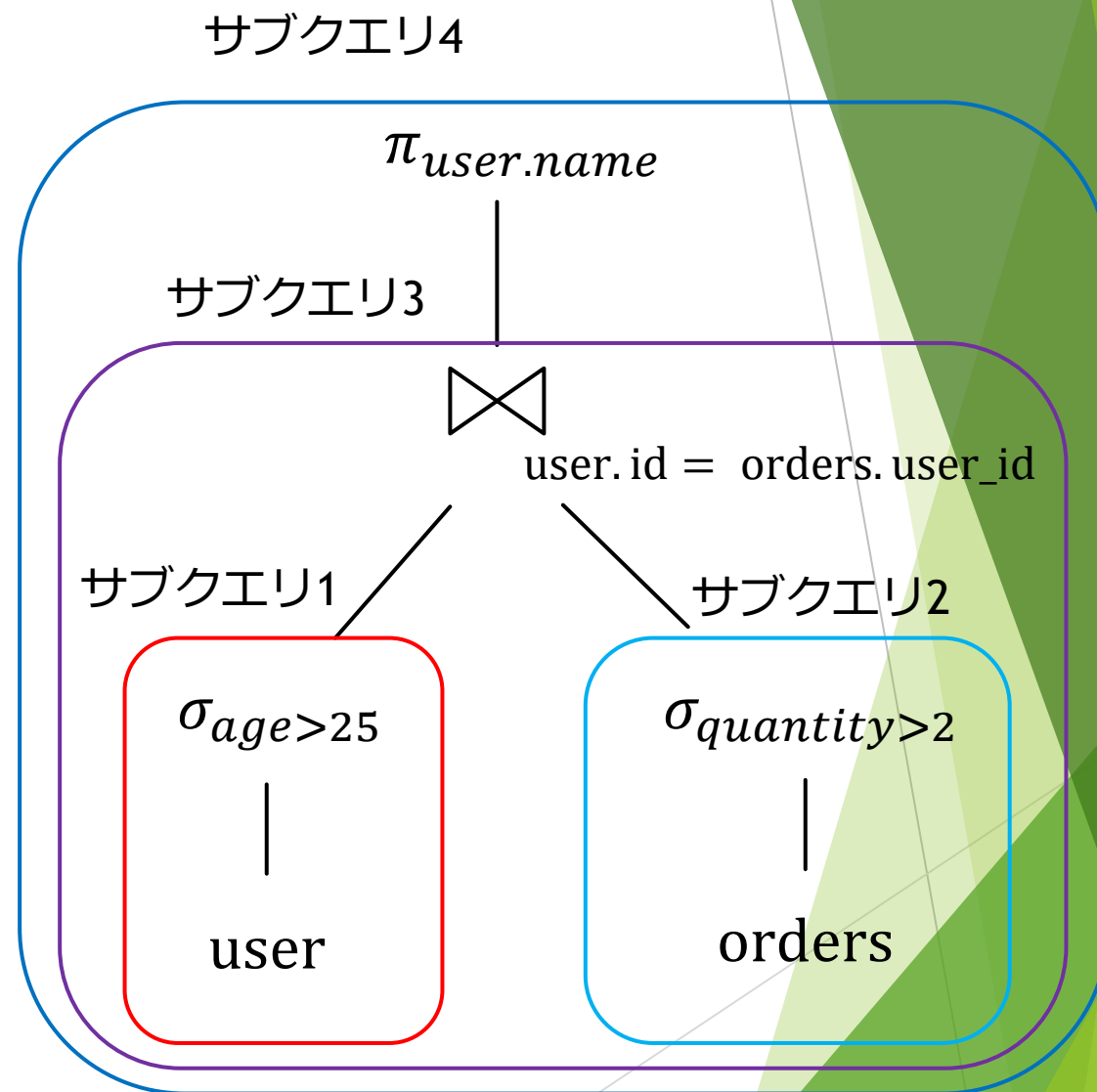
提案手法



提案手法

▶ 実体化ビュー候補列挙

- ▶ ワークロードに含まれる各クエリが包含する全てのサブクエリを列挙する
- ▶ これらが実体化ビューの候補となる

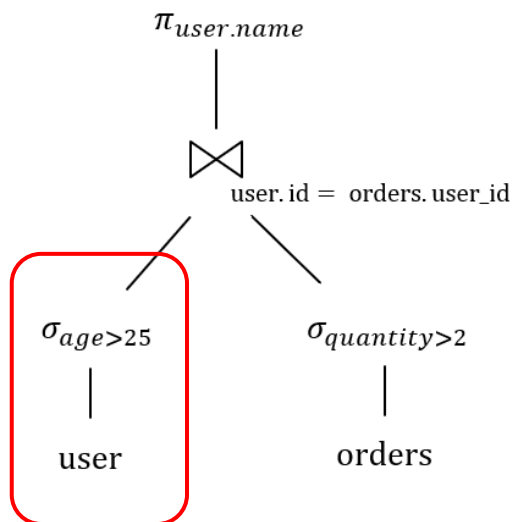


提案手法

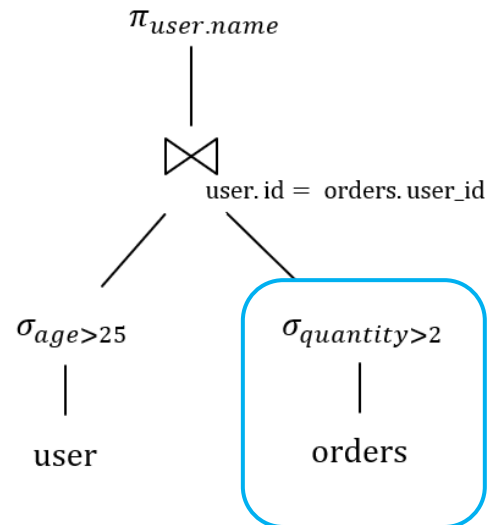
▶ クエリプラン列挙

- ▶ ワークロード内の各クエリについて、どの実体化ビューを用いるかの全ての組み合わせを考え、クエリプランを列挙する

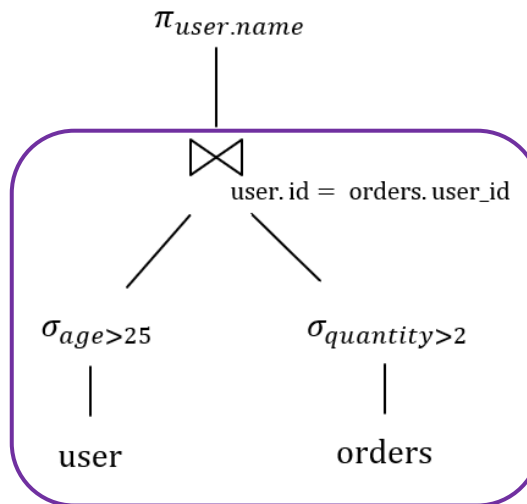
クエリプラン1



クエリプラン2



クエリプラン3

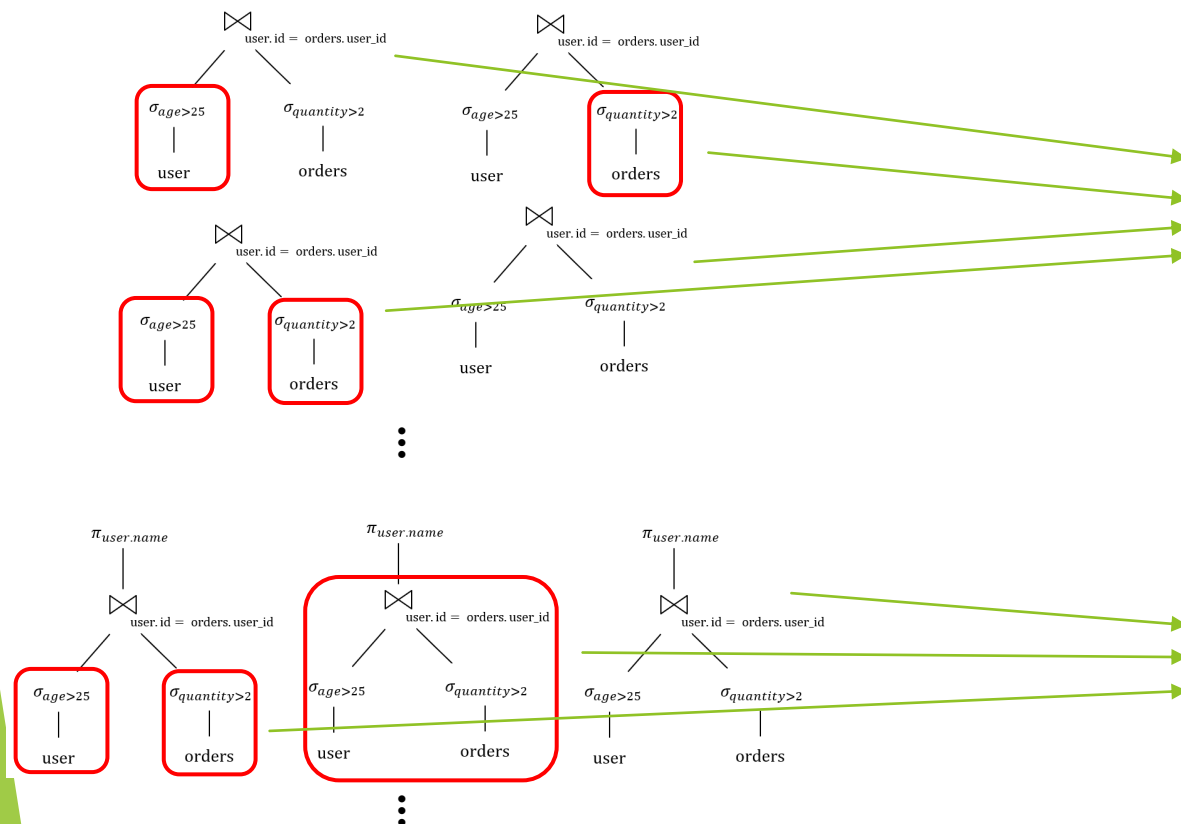


提案手法

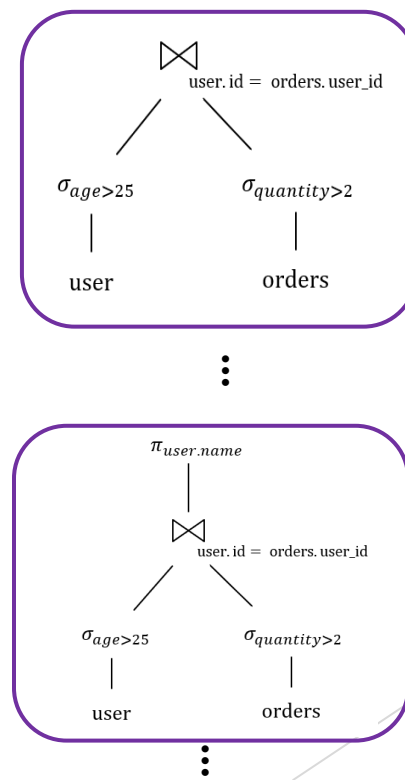
▶ マイグレーションプラン列挙

- ▶ 各実体化ビュー候補について、どのようにしてその実体化ビューを作成するかプランを列挙する

時刻t



時刻t+1



ILPによる定式化

ワークロード実行コストとマイグレーションコストを1つのILPで定式化

Workload Cost

$M = \operatorname{argmin}(\sum_{t=0}^T (\underbrace{\sum_i \sum_j f_i(t) C_{ij} \delta_{ijt}}_{\text{Read Execution Cost}} + \underbrace{\sum_m \sum_n f_m(t) C'_{mn} \delta_{nt}}_{\text{Write Execution Cost}}))$

Schema sequence across time steps

$f_i(t), f_m(t)$: frequency of each operation at time t

Migration Cost

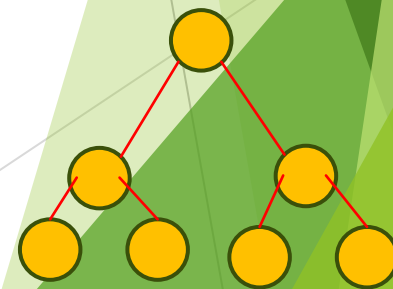
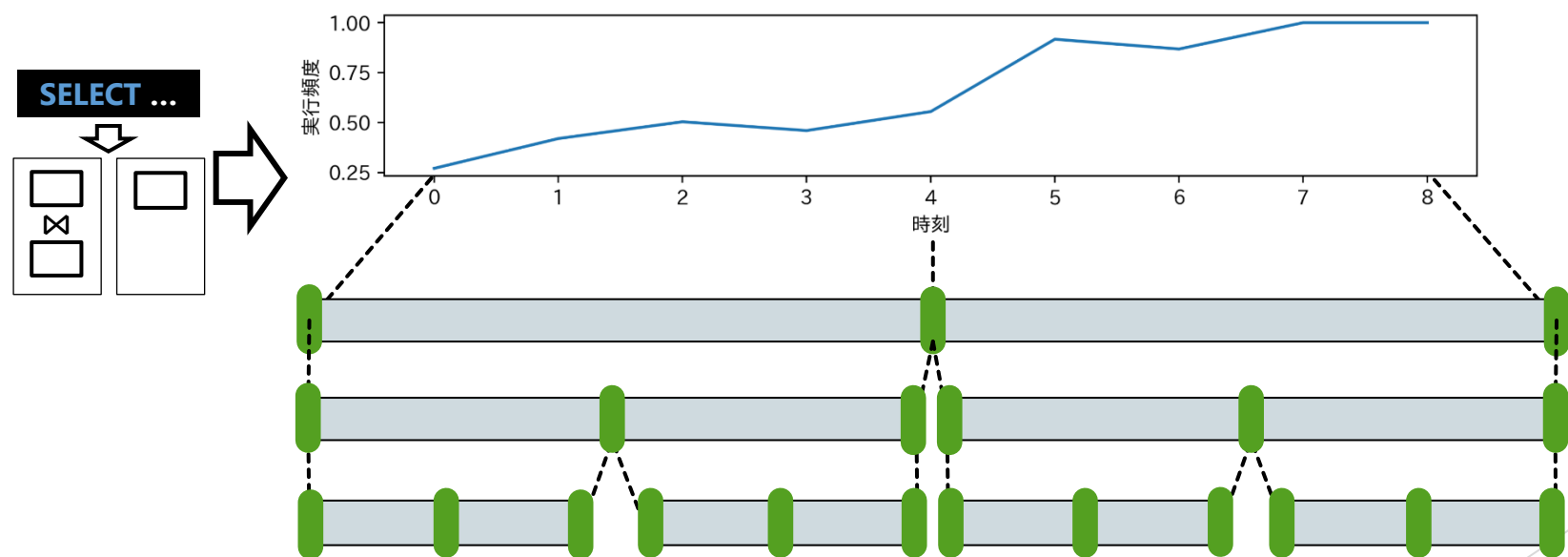
$+ \sum_{t=1}^T (\underbrace{\sum_h C''_h \delta'_{ht}}_{\text{Cost of data selection from old schema}} + \underbrace{\sum_j C'''_j \delta''_{jt}}_{\text{Cost of data insertion into new schema}} + \underbrace{\sum_m f_{mt} \alpha \sum_n C''''_{mn} S_n \delta_{nt}}_{\text{Data maintenance cost for ongoing update operations}})$

問題点: 決定変数(δ_{ijt} , δ_{nt} , δ'_{ht} , δ''_{jt}) は時刻が増えるほど増加し、最適化時間も増加してしまう

プルーニングによるスキーマ候補の削減

再帰的なワークロードの抽象化を活用し、最適化において見込みのない
実体化ビュー候補を削減する

- もとのワークロードを木構造として要約し近似する。
- 各サブワークロードは、3つのタイムステップで単純化する

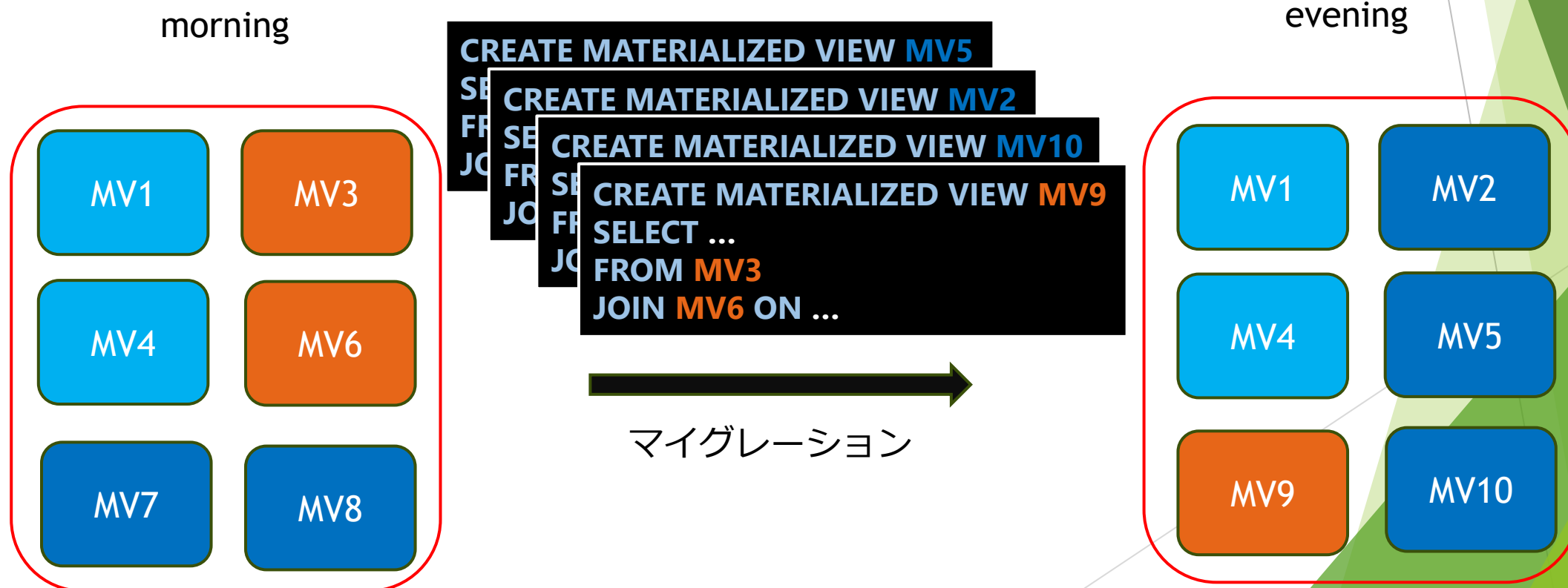


提案手法

- ▶ コスト推定
 - ▶ 最適化の際に必要となる、各クエリプランのコスト、マイグレーションプランのコストをPostgreSQLのEXPLAINコマンドで取得
- ▶ ILPを用いた最適化
 - ▶ プルーニングにより削減された実体化ビュー候補と、各クエリ、マイグレーションプランのコストを使用

現在の進捗

- ▶ 小規模の実験環境において、2つのタイムステップを用意し各時刻においてワークロード実行コストのみを考慮した最適化を実行
- ▶ 時刻間でのマイグレーションプランの取得を完了



今後の方針

- ▶ 今後は、マイグレーションコストを含めた最適化の実装に取り組む予定
 - ▶ 先ずは小規模の実験環境で動作を確認
 - ▶ その後、プルーニングを実装し効果を調べる
- ▶ 大規模実験で用いるデータベースや性能評価指標を考える

ご静聴ありがとうございました

参考文献

- [1] S. Agrawal, S. Chaudhuri, and V. R. Narasayya. Automated Selection of Materialized Views and Indexes in SQL Databases. VLDB, (5):496-505, 2000.
- [2] H. Kimura, G. Huo, A. Rasin, S. Madden, and S. Zdonik. Coradd: Correlation aware database designer for materialized views and indexes. PVLDB, 3:1103-1113, 09 2010
- [3] L. Ma, D. V. Aken, A. Hefny, G. Mezerhane, A. Pavlo, and G. J. Gordon. Querybased Workload Forecasting for Self-Driving Database Management Systems. In SIGMOD, page 631-645, 2018.

